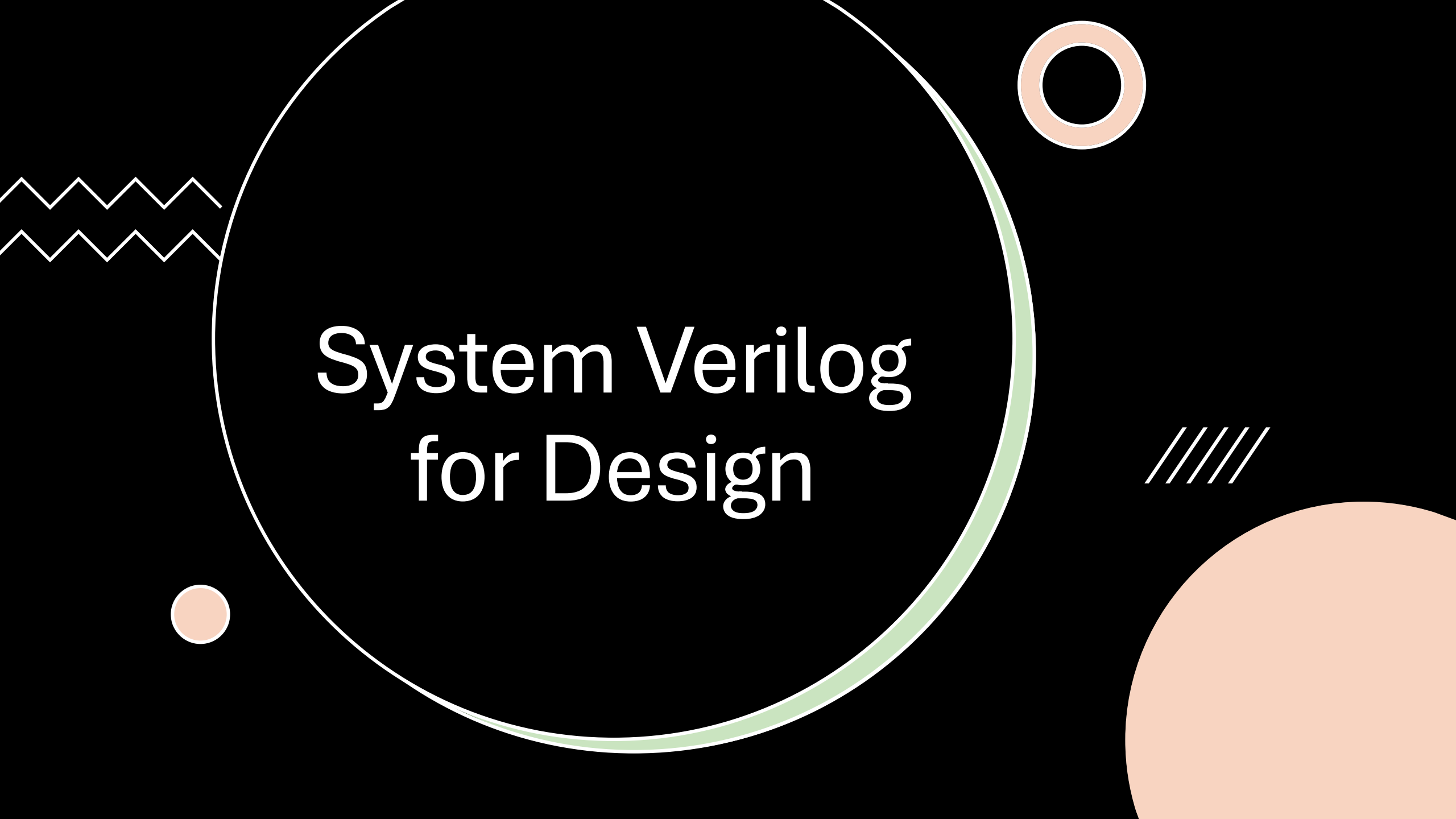




System Verilog for Design

Logistics

- Office hours this week moved to Thursday 3:30 to 4:30pm
- Next week's lectures
 - Will be on Zoom
 - Do not come to the classroom

Assumption: You're familiar with Verilog

HDL vs. Programming Languages (PLs)

- Syntactically similar constructs; Very different semantics
- Statements are evaluated in parallel (generally)
 - In PLs, statements are evaluated serially
- Modules are “instantiated” and connected using wires
 - In PLs, subroutines are “called” and they pass/return values

Think hardware!

Draw hardware first. Then write code.

Ending the Wire vs. Reg Confusion

Verilog

- **wire** for LHS of assign statements
- **reg** for LHS of code inside always @ blocks

```
wire a;  
reg b, c;  
assign a = ____;  
always @(*) b = ____;  
always @(posedge clk) c <= ____;
```

SystemVerilog

- **logic** for LHS of assign statements
- **logic** for LHS of code inside always @ blocks
- logic is 4-state 0,1,x,z

```
logic a, b, c;  
assign a = ____;  
always @(*) b = ____;  
always @(posedge clk) c <= ____;
```

Both: the **context** determines **sequential** or **combinational** logic

Another commonly mentioned new type is “bit”. Two-state logic. Use only in verification.

Signal Your Intent With Specific Always Blocks

Verilog

```
always @(*) begin
    if (x) b = a
    else b = !a;
end
```

```
always @(posedge clk) begin
    if (x) c <= !a;
    else c <= a;
end
```

Coding style is used to verify that c infers as a register and b as comb logic

SystemVerilog

```
always_comb begin
    if (x) b = a
    else b = !a;
end
```

```
always_ff @(posedge clk) begin
    if (x) c <= !a;
    else c <= a;
end
```

New **always_comb** and **always_ff** statements for safety

Provides some type checking. Still need @posedge or @negedge and non-blocking assignments in always_ff to infer sequential logic.

Autoconnect (Implicit Port Connections)

- How many times have you done this?

```
module mod ( input  a, b,  output  c); endmodule
```

```
reg  a, b;  wire  c;  
mod x (.a(a), .b(b), .c(c));
```

- In SystemVerilog, there's a concise **shortcut**

```
mod x ( . * );
```

- Implicit connections only work if port names and widths match

Typedefs (Type Aliases)

- User-defined type
 - typedef is used to expose user-defined types
- They help attach semantic meaning to your design

```
typedef logic [7:0] vec8_t;  
  
input vec8_t op1;  
vec8_t busa, busb;  
function vec8_t checksum;  
...
```

```
// typed (named) type  
typedef logic [15:0] vec16_t;  
vec16_t wordvar;
```

Use Enums Over Parameters

Verilog

```
parameter STATE_IDLE = 2'b00;
parameter STATE_A = 2'b01;
parameter STATE_B = 2'b10;
reg [1:0] state;

always @(posedge clk) begin
    case (state)
        STATE_IDLE: state <= STATE_A;
        STATE_A: state <= STATE_B;
        STATE_B: state <= STATE_IDLE;
    endcase
end
```

SystemVerilog

```
typedef enum logic [1:0] {
    STATE_IDLE, STATE_A, STATE_B
} state_t;
state_t state;
```

Enhance FSM descriptions by
using enums

```
always_ff @(posedge clk) begin
    case (state)
        STATE_IDLE: state <= STATE_A;
        STATE_A: state <= STATE_B;
        STATE_B: state <= STATE_IDLE;
    endcase
end
```

Enums automatically check whether all values can fit.
Add semantic meaning to constants.

More on Enums

- Common to use enums for attaching semantic strings to values

```
typedef enum logic      {  
    READ, WRITE  
} mem_op_t;  
  
module memory (  
    input  [ 4: 0] addr,  
    input  mem_op_t op,  
    input  [31: 0] din,  
    output logic   [31: 0] dout  
);
```

Note that input/output net types are by default 'wire', you can override them as logic

Even More on Enums

- You can force enum values to be associated with a specific value
 - To help match up literals for a port that doesn't use enums

```
typedef enum logic [1:0] { STATE_IDLE=3, STATE_A=2, STATE_B=1 } state_t
```

- You can generate N enum values without typing them out

```
typedef enum logic [1:0] { STATE[3] } state_t  
// STATE0 = 0, STATE1 = 1, STATE2 = 2
```

- You can generate N enum values in a particular range

```
typedef enum logic [1:0] { STATE[3:5] } state_t  
// STATE3 = 0, STATE4 = 1, STATE5 = 2
```

Even More on Enums

- Enums are a first-class datatype in SystemVerilog
 - Enum instances have native functions defined on them
 - next(): next value from current value
 - prev(): previous value from current value
 - num(): number of elements in enum
 - name(): returns a string with the enum's name (useful for printing using \$display)
- They are weakly typechecked
 - You can't assign a binary literal to an enum type net
- They show up in waveforms
 - No more confusion trying to correlate literals to a semantic name

Structs

- Allows designer to group nets together, helps encapsulation of signals, easy declaration
- Can be used within a module or in a module's ports

```
typedef struct { // structure definition
    logic [31:0] a, b;
    logic [ 7:0] opcode;
    logic [23:0] address;
} instruction_word_t;

instruction_word_t IW; // structure allocation
```

```
module alu (input instruction_word_t IW, clock);
```

```
module processor (...);
    ...
    typedef enum {ADD, SUB, MULT, DIV} opcode_t;
    typedef struct { // typedef is local
        logic [31:0] a, b;
        opcode_t      opcode;
        logic [23:0] address;
        logic          error;
    } instruction_word_t;

    function alu (input instruction_word_t IW);
        ...
    endfunction
endmodule
```

Multidimensional Arrays

- **Packed** dimensions are to the **left** of the variable name
 - Packed arrays are used when you want to treat a group of bits as a single word or vector (e.g., for arithmetic or bitwise operations).
- **Unpacked** dimensions are to the **right** of the variable name
 - Unpacked arrays are used when you want to treat each element as an independent variable (e.g., an array of registers or signals).
- SystemVerilog allows multiple packed and unpacked dimensions

```
bit [0:3] [7:0] packedArray ;
```

```
int Array[8][32];
```

[illegible]

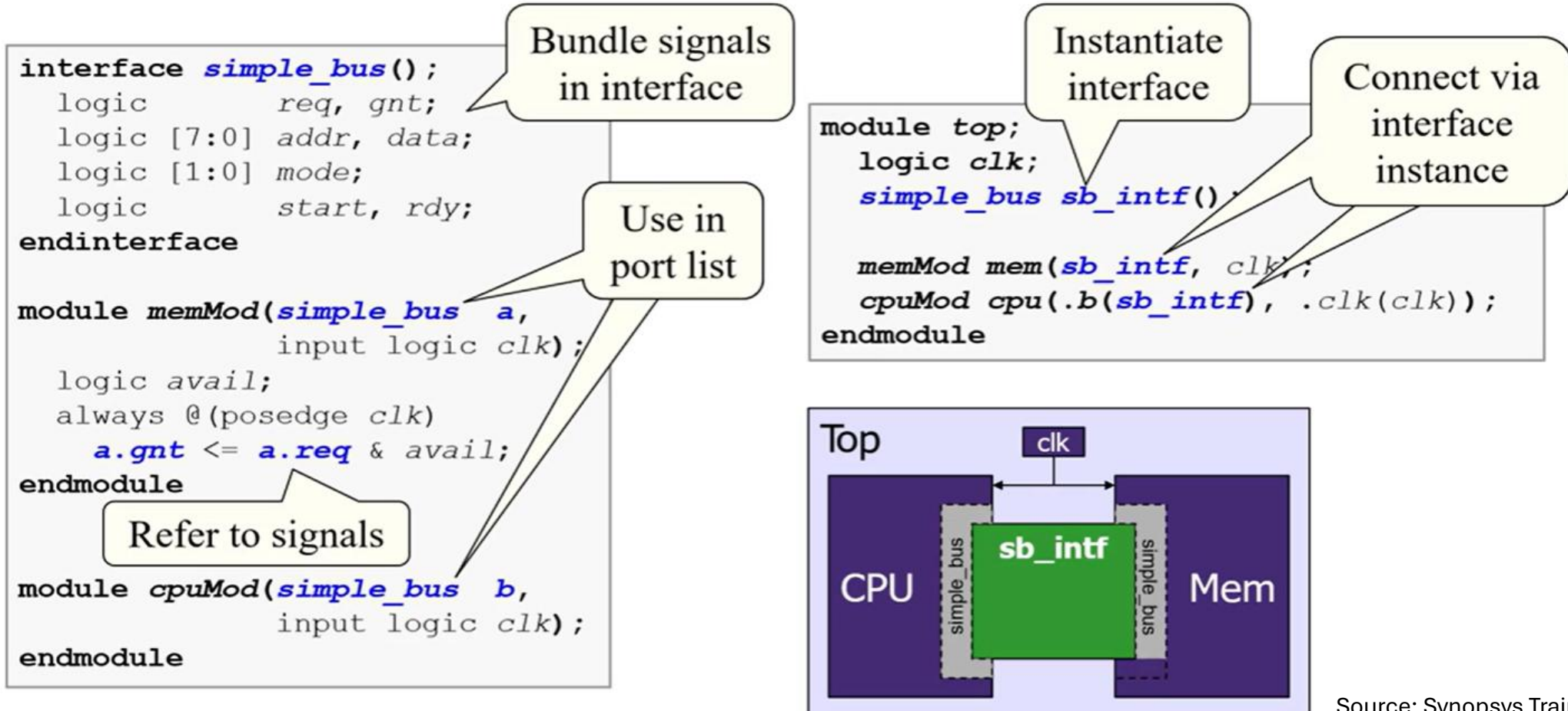
```
bit [1:5] [1:6] v4 [1:7] [1:8]; // 1 to 6 varies most rapidly, followed by  
// 1 to 5, then 1 to 8 and then 1 to 7
```

Multidimensional Arrays

- Typically only 1 packed and 1 unpacked dimensions are used in designs
- How to access individual bits?
 - Declaration:
 - `logic [packed_dimensions] name [unpacked_dimensions];`
 - `//logic [bits] name [elements];`
 - Access:
 - `Name[unpacked_index][packed_index]`
 - Example:
 - `logic [3:0] t[6];`
 - `t[array_index][bit_index]`
 - `t[5][3]` refers to the 3rd bit of the 5th element

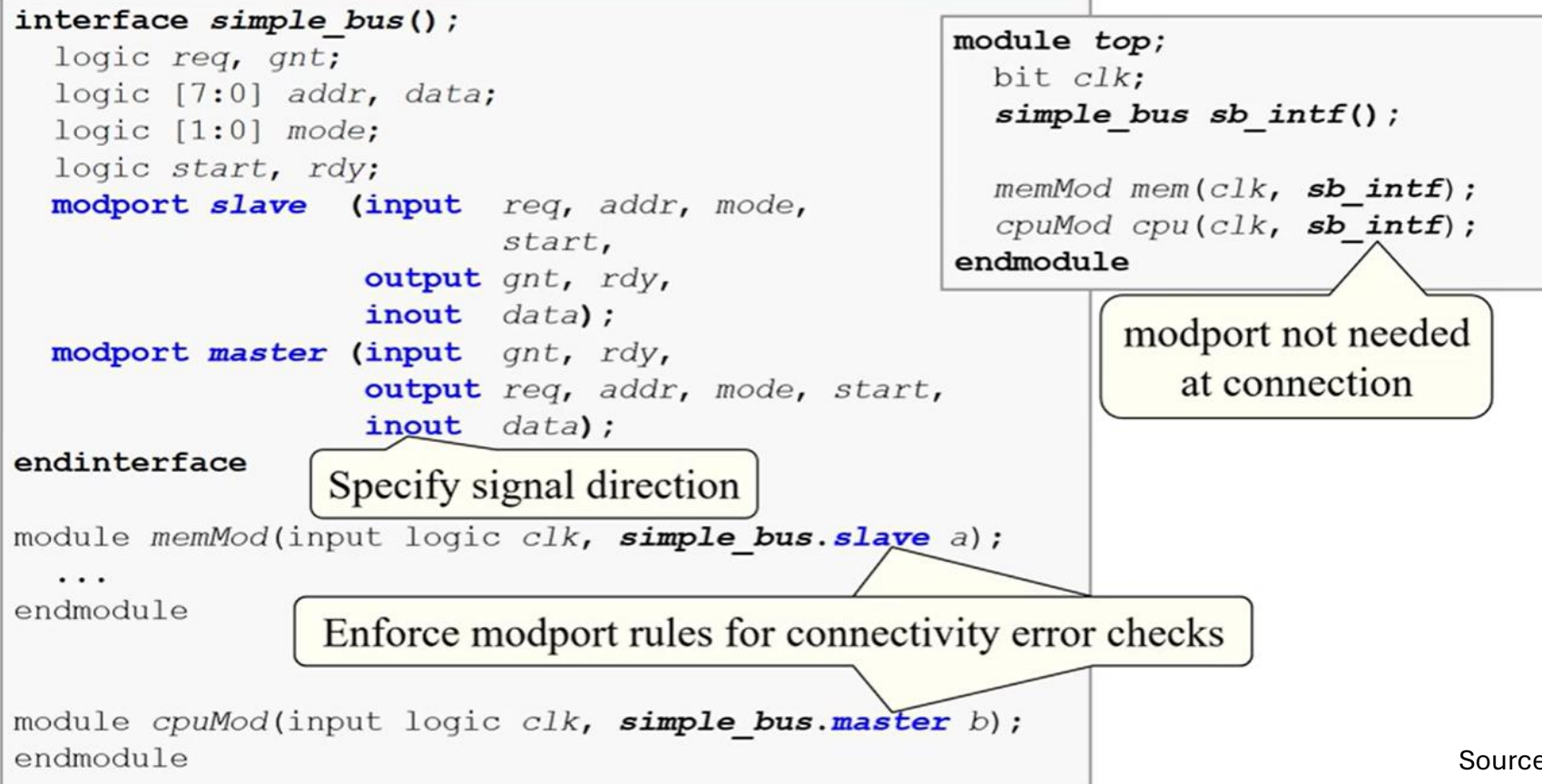
Interfaces

- Interfaces allow designers to group ports together
 - **Simplify connections** between parent and child modules



Modports

- But I didn't specify the direction (input/output) of the interface ports!
 - This can cause multi-driver issues with improper connections
- Solution: use modports



Packages / Namespacing

- Verilog has a global namespace
 - Often naming conflicts in large projects
 - ``include` is hacky and requires ``ifdef` guards
- SystemVerilog allows you to encapsulate constructs in a package
 - modules, functions, structs, typedefs, classes

```
package my_pkg;
    typedef enum logic [1:0] { STATE[4] } state_t;
    function show_vals();
        state_t s = STATE0;
        for (int i = 0; i < s.num; i = i + 1) begin
            $display (s.name());
            s = s.next();
        end
    endfunction
endpackage
```

```
import my_pkg::* ;

module ex ( input clk);
    state_t s;
    always_ff @(posedge clk) begin
        s <= STATE0;
    end
endmodule
```

Unsize literals

- Verilog had unsized literals
 - `x = 0;`
 - Treated as 32-bit value
- SV adds intelligent unsized literals
 - `x = '0;`
 - Aware of the bit-width of the signal
 - Fills target with that single value

Verilog

```
// Sized literal
// <size>'<base><value>

logic [5:0] databus;

databus = 6'b0;    // 000000
databus = 6'b1;    // 000001
databus = 6'bz;    // zzzzzz
databus = 6'bx;    // xxxxxx
databus = 4'bx;    // 00xxxx
```

SystemVerilog

```
// Unsized literal
// '<value>

logic [5:0] databus;

databus = '0;    // 000000
databus = '1;    // 111111
databus = 'z;    // zzzzzz
databus = 'x;    // xxxxxx
```

Enhanced Timescale Management

- New keywords introduced
 - timeunit
 - timeprecision
- Equivalent to `timescale but without file order compilation dependency
- timeunit and timeprecision are
 - Declarations, not directives
 - Placed inside a module
 - Only visible to the design element where they are declared and any nested units

```
module testbench;  
    timeunit 1ns;  
    timeprecision 100ps;  
    logic status;  
    initial begin  
        #10ns status = 1'b1;  
        #10 status = 1'b0;  
    end  
endmodule
```

```
module testbench2;  
    timeunit 1ns/100ps;  
    logic status;  
  
    initial begin  
        #10ns status = 1'b1;  
        #10 status = 1'b0;  
    end  
endmodule
```

Tasks and Functions

- Several syntactical improvements

```
task automatic cpu_driver_ref
(input logic [7:0] write_data,
 ref logic data_read,
 ref logic data_valid,
 ref logic [7:0] cpu_data);

#5 data_valid = 1;
wait (data_read == 1);
#20 cpu_data = write_data;
wait (data_read == 0);
#20 cpu_data = 8'hzz;
data_valid = 0;

endtask : cpu_driver_ref
...
cpu_drive_ref(8'hff, req, ack, data);
```

All output
updates seen

Input changes detected

Formal arguments
mapped to actuals in call

```
function logic [15:0] square(logic [7:0] x);
  return x * x;
endfunction
```

```
function [7:0] addcarry (input [7:0] a, b,
  output carry);
  {carry, addcarry} = a + b;
endfunction

logic [7:0] a, b, sum;
logic cry;

initial begin
  sum = addcarry(a, b, cry);
  ...
```

cry is an output

Enhancements to Generate Statements

- “generate” keyword optional
- Can use “i++” instead of “i=i+1”

```
// The for-loop creates 16 assign statements
genvar i;
for (i=0; i < 16; i++) begin
    assign temp[i] = (select == i) ? mux_in[i] : 0;
end
```

Guidelines

Simple combinational Logic

```
logic D;  
assign D = A || B;
```

Complex combinational Logic

```
logic D;  
always_comb  
begin  
    if (C) D = A  
    else D = B;  
end
```

Sequential Logic

```
logic Q;  
always_ff @(posedge clk)  
begin  
    Q <= D;  
end
```

Combined combinational and sequential Logic

```
logic Q;  
always_ff @(posedge clk)  
begin  
    Q <= A || B;  
end
```

Testbenches

```
bit D;  
initial begin  
    #10 D = A || B;  
end
```

It's not a **program**, it's a **design**.
You're not **calling**. You're **instantiating**.

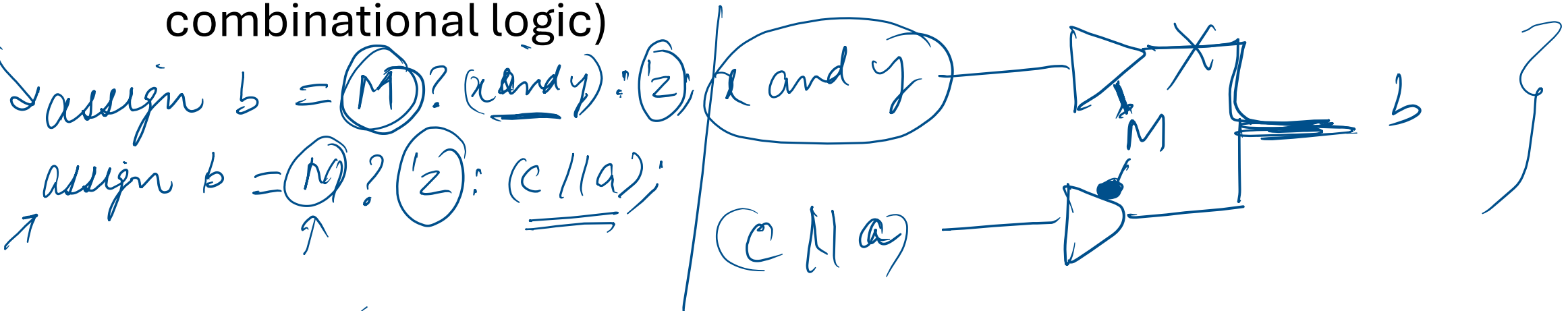
Guidelines

- Do not mix blocking and non-blocking assignments in the same always block
- Always use a reset signal (sync or async) to initialize flops
- No multiple drivers (unless explicitly driving a wire with “bus” semantics)
- Avoid latch inference (define all cases in if/case statements for combinational logic)

It's not a **program**, it's a **design**.
You're not **calling**. You're **instantiating**.

Guidelines

- Do not mix blocking and non-blocking assignments in the same always block
- Always use a reset signal (sync or async) to initialize flops
- No multiple drivers (unless explicitly driving a wire with “bus” semantics)
- Avoid latch inference (define all cases in if/case statements for combinational logic)



Building Blocks

- Combinational Logic
 - Gates, Multiplexer, Decoder
- Sequential Logic
 - FF, Register, Counter
- Control Logic
 - FSM
- Arithmetic
 - Adder, Multiplier
- Memories
 - Single-port, Dual-port memory
 - Clocked read, Combinational read
- Interconnect
 - Crossbar

Multiplexer

```
module mux4to1 (  
    input logic [1:0] sel,    // 2-bit select line  
    input logic    in0,    // Input 0  
    input logic    in1,    // Input 1  
    input logic    in2,    // Input 2  
    input logic    in3,    // Input 3  
    output logic    out    // Output  
);  
    always_comb begin  
        case (sel)  
            2'b00: out = in0;  
            2'b01: out = in1;  
            2'b10: out = in2;  
            2'b11: out = in3;  
            default: out = 1'b0; // Optional: default case  
        endcase  
    end  
endmodule
```

Counter

```
module counter (  
    input logic    clk,    // Clock signal  
    input logic    rst_n,  // Active-low asynchronous reset  
    input logic    clear,  // Synchronous clear  
    input logic    load,   // Synchronous load  
    input logic    enable, // Enable counting  
    input logic [3:0] data_in, // Data to load  
    output logic [3:0] count // Counter output  
);  
    always_ff @(posedge clk or negedge rst_n) begin  
        if (!rst_n)  
            count <= 4'b0000; // Asynchronous reset  
        else begin  
            if (clear)  
                count <= 4'b0000; // Synchronous clear  
            else if (load)  
                count <= data_in; // Load data  
            else if (enable)  
                count <= count + 1; // Increment counter  
        end  
    end  
endmodule
```

FSM

```
module fsm (  
    input logic clk,  
    input logic rst_n,    // Active-low reset  
    input logic in,       // Input signal  
    output logic [1:0] out // 2-bit output  
);  
    // Define state encoding  
    typedef enum logic [1:0] {  
        IDLE = 2'b00,  
        S1   = 2'b01,  
        S2   = 2'b10,  
        S3   = 2'b11  
    } state_t;  
    state_t current_state, next_state;  
    // State transition logic  
    always_ff @(posedge clk) begin  
        if (!rst_n)  
            current_state <= IDLE;  
        else  
            current_state <= next_state;  
    end
```

```
    // Next state logic  
    always_comb begin  
        case (current_state)  
            IDLE: if (in) next_state = S1 else next_state = IDLE;  
            S1:  if (in) next_state = S2 else next_state = IDLE;  
            S2:  if (in) next_state = S3 else next_state = S2;  
            S3:  if (in) next_state = S3 else next_state = IDLE;  
            default: next_state = IDLE;  
        endcase  
    end  
    // Output logic  
    always_comb begin  
        case (current_state)  
            IDLE: out = 2'b00;  
            S1:  out = 2'b01;  
            S2:  out = 2'b10;  
            S3:  out = 2'b11;  
            default: out = 2'b00;  
        endcase  
    end  
endmodule
```

Adder

```
module full_adder (  
    input logic a, b, cin,  
    output logic sum, cout  
);  
    assign sum = a ^ b ^ cin;  
    assign cout = (a & b) | (b & cin) | (a & cin);  
endmodule
```

```
module ripple_carry_adder(  
    input logic [3:0] a, b,      // 4-bit inputs  
    input logic cin,           // Initial carry-in  
    output logic [3:0] sum,     // 4-bit sum output  
    output logic cout          // Final carry-out  
);  
    logic [4:0] carry;          // Carry chain (5 bits: cin + 4 internal carries)  
    assign carry[0] = cin;  
    genvar i;  
    for (i = 0; i < 4; i++) begin : adder_stage  
        full_adder fa (  
            .a (a[i]),  
            .b (b[i]),  
            .cin (carry[i]),  
            .sum (sum[i]),  
            .cout (carry[i+1])  
        );  
    end  
    assign cout = carry[4]; // Final carry-out  
endmodule
```

Memory

```
module single_port_memory #(
    parameter DATA_WIDTH = 8,
    parameter ADDR_WIDTH = 8
)(
    input logic          clk,
    input logic          we,    // Write enable
    input logic [ADDR_WIDTH-1:0] addr, // Address
    input logic [DATA_WIDTH-1:0] din,  // Data input
    output logic [DATA_WIDTH-1:0] dout // Data output
);
    // Memory array
    logic [DATA_WIDTH-1:0] mem [0:(1<<addr_width)-1];

    // Write logic (synchronous)
    always_ff @(posedge clk) begin
        if (we)
            mem[addr] <= din;
        end
    // Read logic (combinational)
    assign dout = mem[addr];
endmodule
```

Memory using Interfaces

```
interface mem_if #(parameter ADDR_WIDTH = 4,  
DATA_WIDTH = 8);
```

```
    logic clk;
```

```
    logic we;
```

```
    logic [ADDR_WIDTH-1:0] addr;
```

```
    logic [DATA_WIDTH-1:0] din;
```

```
    logic [DATA_WIDTH-1:0] dout;
```

```
    // Modport for memory module
```

```
    modport mem_side (
```

```
        input  clk, we, addr, din,
```

```
        output dout
```

```
    );
```

```
    // Modport for testbench
```

```
    modport tb_side (
```

```
        output clk, we, addr, din,
```

```
        input  dout
```

```
    );
```

```
endinterface
```

```
module single_port_mem (  
    mem_if.mem_side mem  
);
```

```
    parameter DEPTH = 1 << mem.ADDR_WIDTH;
```

```
    logic [mem.DATA_WIDTH-1:0] mem_array [0:DEPTH-1];
```

```
    always_ff @(posedge mem.clk) begin
```

```
        if (mem.we)
```

```
            mem_array[mem.addr] <= mem.din;
```

```
            mem.dout <= mem_array[mem.addr];
```

```
    end
```

```
endmodule
```

Memory using Interfaces

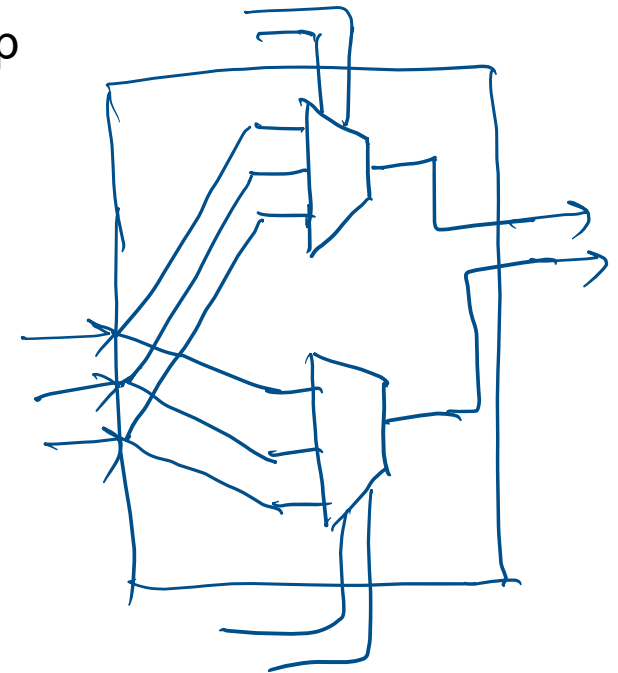
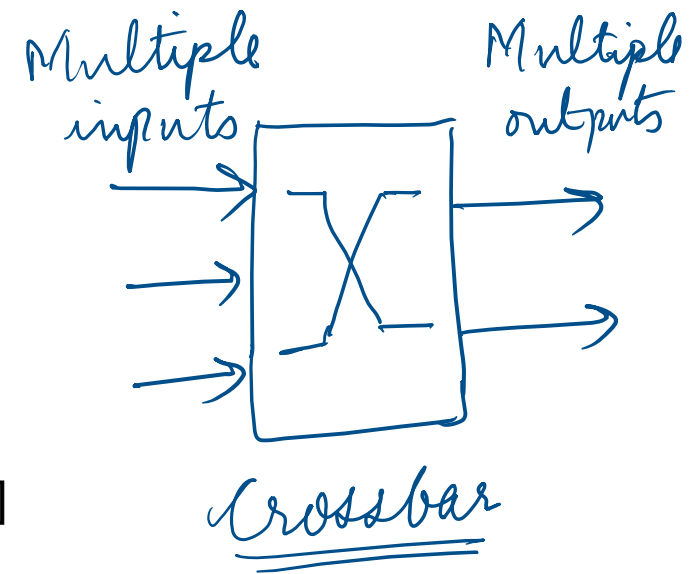
```
module tb(  
    mem_if.tb_side mem  
);  
    // Clock generation  
    initial mem.clk = 0;  
    always #5 mem.clk = ~mem.clk;  
    initial begin  
        // Write to address 3  
        mem.we = 1;  
        mem.addr = 4'd3;  
        mem.din = 8'hA5;  
        #10;  
        // Read from address 3  
        mem.we = 0;  
        mem.addr = 4'd3;  
        #10;  
        $display("Read data: %h", mem.dout);  
        $finish;  
    end  
endmodule
```

```
module top_tb;  
    // Instantiate the interface  
    mem_if #(4, 8) mem_bus();  
  
    // DUT instantiation  
    single_port_mem dut (  
        .mem(mem_bus)  
    );  
  
    //Test instantiation  
    tb u_tb(  
        .mem(mem_bus)  
    );  
endmodule
```

Crossbar

```
import crossbar_pkg::*;

module crossbar (
    input logic [DATA_WIDTH-1:0] in [N],
    input logic [N-1:0] sel [M],
    output logic [DATA_WIDTH-1:0] out [M]
);
    genvar i, j;
    generate
        for (i = 0; i < M; i++) begin : output_loop
            always_comb begin
                out[i] = '0;
                for (j = 0; j < N; j++) begin
                    if (sel[i][j])
                        out[i] = in[j];
                end
            end
        end
    endgenerate
endmodule
```



```
package crossbar_pkg;
    parameter int DATA_WIDTH = 8;
    parameter int N = 4; // Number of inputs
    parameter int M = 4; // Number of outputs
endpackage
```

Reference Material

- LRM
 - [1800-2023 - IEEE Standard for SystemVerilog--Unified Hardware Design, Specification, and Verification Language | IEEE Standard | IEEE Xplore](#)
- SV for Design book
 - [SystemVerilog for Design Second Edition: A Guide to Using SystemVerilog for Hardware Design and Modeling | SpringerLink](#)

Synopsys Trainings

- Please complete this training: [“SystemVerilog for RTL Design”](#)

A good review

- <https://compas.cs.stonybrook.edu/~nhonarmand/courses/sp18/cse502/slides/03-systemverilog.pdf>